<- □□□

Mutex starvation in Go

2021/04/07

Introduction

Mutex is one of the synchronization primitives in multi-threaded programming. A mutex object usually has a `Lock()` `Unlock()` interface and can lock/unlock itself. Also, it guarantees that it cannot be locked by multiple threads at the same time. If the user program always acquires a lock before entering a critical section, and releases the lock as soon as the critical section is over, "mutual exclusions" can be performed even in a multi-threaded environment.

Since mutex is a very simple locking mechanism, there are many variations in its algorithm. However, if the algorithm is not carefully chosen, it can sometimes cause "starvation", resulting in reduced throughput and even system stop. In this article, we will discuss the mutex algorithm that causes starvation in certain situations that existed up to Go 1.8. We will use actual programs as examples to gain a better understanding of the mutex behavior, starvation, and locking algorithms. Then, we will look at the changes that Go has made in 1.9 to make situation better.

Mutex "unfairness" in Go1.8

The `sync` section in the `Minor changes to the library` of the release notes for Go1.9 says `Mutex is now more fair .`

First, let's look at a problem that existed in the mutex up to Go1.8. This happens in a situation like: "two goroutines are fighting for a lock, one holds it for a long time and releases it for a short time, the other holds it for a short time and releases it for a long time. The code just looks like this:

```
package main

import (
    "sync"
    "time"
)

func main() {
    done := make(chan bool, 1)
    var mu sync.Mutex

    // goroutine 1
    go func() {
```

```
        for {
            select {
            case <-done:
                return
            default:
                mu.Lock()
                time.Sleep(100 * time.Microsecond)
                mu.Unlock()
            }
        }
    }()

    // goroutine 2
    for i := 0; i < 10; i++ {
        time.Sleep(100 * time.Microsecond)
        mu.Lock()
        mu.Unlock()
    }
    done <- true
}
```

This program starts one goroutine (*goroutine 1*), and inside it, it starts a for-loop. In the loop, it locks the mutex, sleeps for 100µs, and then releases the lock. In another goroutine (*goroutine 2*), it starts a for-loop that runs only 10 times, and in the loop, it sleeps for 100µs, then acquires the lock, and immediately unlocks it.

Now let's think about how this program will work. My guess is: "goroutine2 will eventually acquire the lock 10 times, but in the meantime, goroutine1 will also acquire the lock several times, and the program will terminate normally soon, probably in no longer than 1 or 2 seconds". When I tried to run this program with Go1.8 in my local environment, I found that the program did not finish at all. It took so long that I had to quit the program in the middle, but the program did not finish for at least 3 minutes. (If you are interested, you can give it a try.)

If you have Go1.9 or later, this program terminates immediately as expected. Before the explaination why this happens, there are a few things I want to write down first.

Starvation

In the context of multitasking, starvation means that a process is unable to obtain the resources it needs for a long period of time. In the context of mutexes, stavation is more specifically defined as a situation where a process is unable to acquire READ/WRITE locks. In the above program, stubation is occurring: goroutine2 wants to acquire a lock, but it is not able to do so because of the mutex algorithm described below. In general, the acquisition of locks between threads is expected to be as fair as possible keeping the throughput.

Algorithm that determines the behavior of mutexes

As mentioned above, mutex is a simple synchronization primitive. Let's check its behavior again.

- A mutex that is not locked can be locked with `Lock()`.
- A mutex that is locked can be unlocked with `Unlock()`.
- Only one thread can lock at a time.
- When trying `Lock()` a locked mutex, it will block

There are several locking algorithms that follow this behavior.

The part which these locking algorithms specify in the whole locking process is, which thread will be able to acquire the next lock when there is a lock acquisition contention. Some of them are described below, but this is not exhaustive.

Locking Algorithm #1: Thunder lock

In Thunder Lock, if a thread tries to acquire a lock, but it is already locked, the thread goes to a "pool" of threads waiting for the lock to be released, then sleeps. This is often referred to as "parking". When the lock is released, it wakes up all the threads in the parking pool at once, and they try to acquire the lock again. Of course, only one thread will then acquire the lock, and the others will go to the pool, and sleep again. Thunder Lock causes a Thundering herd when the mutex is released. The advantage of Thunder lock is that it's simple to implement, but I personally have the impression that there is no particular reason to adopt it.

Locking Algorithm #2: Barging

From my brief research, Barging is the term introduced in Java at first. Barge should be understood as "a bridge that carries something from one point to another".

In Barging, usually there is a queue of threads for mutexes waiting the lock release. When a thread tries to lock a locked mutex, it is pushed to the queue and goes to sleep. It sounds similar to Thunder lock.

When the mutex is released, it wakes up the thread that is popped from the queue. However, the lock is not necessarily given to that thread. The lock is passed to either the thread that was woken up or to a thread that is not yet in the queue but happens to be trying to get the lock at the time. To which one depends on the moment. It means that even though there is already a thread waiting to acquire the lock, a new thread which comes just now can acquire a lock. This is called "lock stealing". This is similar to a situation where you are in a queue, but you are interrupted.

Because of lock stealing, and Barging can achieve relatively high throughput. Usually, waking up a sleeping thread is a high-cost operation and very time-consuming, so it is benefical that Barging don't necessarily have to do it. The disadvantages of Barging are that it can lose the fairness of lock acquisition, and that starvation can occur.

Locking Algorithm #3: Handoff

Handoff pops a thread from the queue when the mutex is released, and wakes it up. It does not pass the lock to a new lock acquisition request as in Barging, and the thread at the top of the lock queue always gets the next lock. In other words, Handoff lock is a strict FIFO lock.

Handoff generally cannot achieve high throughput because the thread at the head of the queue must always be woken up to acquire the lock, even if there are threads which can immediately get the lock outside the queue. The thread that wants to acquire the lock must enter the queue and wait for its turn. A patch that implements a Handoff-like mutex in the Linux kernel is here c4296e812c70493bfdc999b5c1c).

Adaptive lock and spin

Now that we have briefly looked at some locking algorithms, they are sometimes used in combination with something else, which is often called adaptive locking. The term "adaptive lock" has no clear definition, and is often used as a generic term for various "combined" locking algorithms. For example, Adaptive Locks: Combining Transactions and Locks for Efficient Concurrency deals with an approach that combines mutexes and STM.

One of the common patterns of adaptive locks is to combine spinning and parking, where the thread spins a few times, and if the lock is still not obtained, the thread is parked. Short spinning is intended to get the lock while spinning in situations like a thread can't get the lock now, but I'll get it soon - specifically, in less time than it takes to park and wake the thread.

For example, from WebKit source code, you can see that it spins 40 times, and if the lock can't be acquired, it perks up.

Algorithm of sync.Mutex in Go1.8

Let's go back to Go. Mutex up to Go1.8 was a combination of the above Barging and Adaptive lock. When it tries to acquire a lock on a locked mutex, it first call sync_runtime_canSpin in mutex.Lock() to determine if it should spin. According to it, the mutex of Go spins only 4 times first.

The algorithm works without taking care of starving goroutines. Because of this implementation, the previous program did not work as expected. I will explain how this has been changed in 1.9.

Algorithm of sync.Mutex in Go1.9

The commit for the fix is this.

In sync/mutex.go in Go1.9, a new concept called "starvation mode" was introduced.

In Go1.9, if a gouroutine was unable to acquire a lock for 1e6ns (= 1ms), it gets starvation mode. The code looks like the following.

```
starving = starving || runtime_nanotime()-waitStartTime > starvationThresholdNs
```

A goroutine in starvation mode will pass `lifo=true` to call a function to acquire the semaphore that the mutex is using internally.

```
// If we were already waiting before, queue at the front of the queue.
queueLifo := waitStartTime != 0
if waitStartTime == 0 {
    waitStartTime = runtime_nanotime()
}
runtime_SemacquireMutex(&m.sema, queueLifo)
```

On the semaphore (in the runtime package) side, if `lifo=true` is passed, the Treap (the data structure that is the substance of the queue) that stores the waiting goroutine places the goroutine at the head of the queue. It means this goroutine overtakes all other goroutines in the queue.

Also, when the mutex is released, the semaphore is released passing handoff=true. On the semaphore side, a goroutine is dequeued from the queue of waiting goroutines at the time of release, and if `handoff=true`, the `ticket` of the dequeued goroutine is directly set to `1`, which means it is [given the right to leave the queue](https://github.com/golang/go/commit/0556e26273f704db73df9e7c4c3d2e8434dec7be#diff-bb07e8d113e0257192c87f8b6153be1bcb547aa7826db102178ce2e6b7fd98d8R178-R195). This means that the goroutine now passes its ownership directly to the goroutine in the queue, without getting the lock stolen by the newly arrived goroutine.

```
s, t0 := root.dequeue(addr)
if s != nil {
    atomic.Xadd(&root.nwait, -1)
}
unlock(&root.lock)
if s != nil {
    // ...
    if handoff && cansemacquire(addr) { // ! here
        s.ticket = 1
    }
    // ...
}
```

These changes mean that goroutines that enter starvation mode can now have their locks passed to them promptly and explicitly, without being intercepted.

Summary

We have looked at the potential problems of mutex stabalization that existed up to Go1.8. As you can see from the comments in the code, Handoff is not as good as Barging in terms of performance, but it provides more fairness. In the original issue, there is an interesting discussion about whether something like `sync.FairMutex` would be better, so I'd recommend readers who are interested to read it.

Reference (articles/papers)

- runtime: fall back to fair locks after repeated sleep-acquire failures
- Go 1.9 Release Notes - Minor changes to the library
- Locking in WebKit
- Fine-grained Adaptive Biased Locking
- (ja) GNU/Linux □□□□□□□□□□□□□ NPTL (Native POSIX Thread Library) Programming.

References (source code)

- https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/commit/?id=9d659ae14b545c4296e812c70493bfdc999b5c1c
- https://github.com/WebKit/WebKit/blob/88278b55563e5ccdc0b3419c6c391c3becc19e40/Source/WTF/wtf/LockAlgorithmInlines.h#L57-L62
- https://trac.webkit.org/browser/trunk/Source/WTF/benchmarks/LockSpeedTest.cpp?rev=200444
- https://trac.webkit.org/browser/webkit/trunk/Source/WTF/wtf/ParkingLot.cpp?rev=200444
- https://trac.webkit.org/browser/trunk/Source/WTF/wtf/Lock.cpp?rev=200444